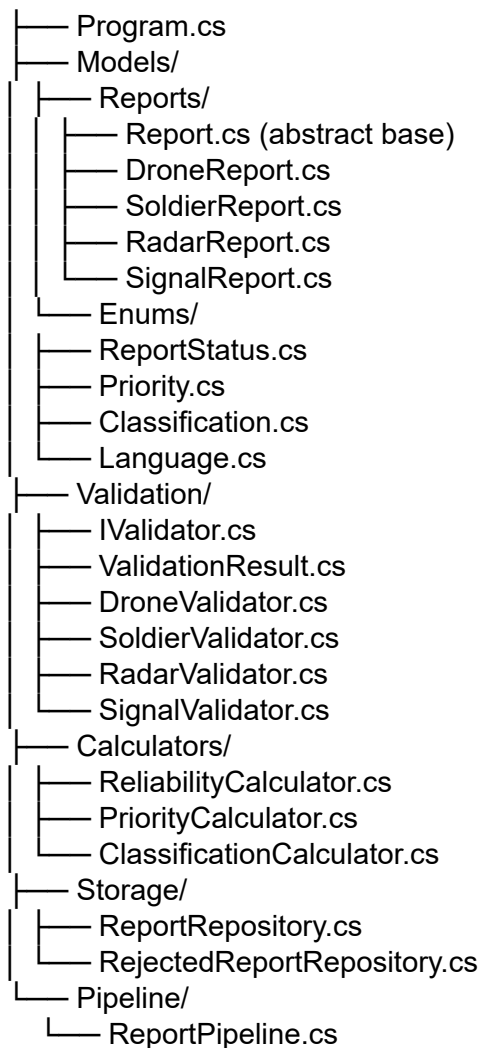


< Intelligence Pipeline System - Reference Architecture

Project Structure

IntelligencePipeline/



Namespaces

- `IntelligencePipeline` - Program entry point
- `IntelligencePipeline.Models.Reports` - Report hierarchy -
- `IntelligencePipeline.Models.Enums` - Enumerations -
- `IntelligencePipeline.Validation` - Validation components -
- `IntelligencePipeline.Calculators` - Calculation logic -
- `IntelligencePipeline.Storage` - Data storage -
- `IntelligencePipeline.Pipeline` - Processing pipeline

Enumerations

ReportStatus

public enum ReportStatus

```
{  
  
    New,  
  
    Validating,  
  
    Validated,  
  
    Rejected,  
  
    InProgress,  
  
    Completed  
}
```

Represents the current state of a report in the processing workflow.

Priority

public enum Priority

```
{  
    Low,  
  
    Medium,  
  
    High,  
  
    Critical  
}
```

Represents the urgency level of a report.

Classification

public enum Classification

```
{  
  
    Unclassified,  
  
    Restricted,  
  
    Secret,  
  
    TopSecret  
  
}
```

Represents the security classification level.

Language

public enum Language

```
{  
  
    Hebrew,  
  
    Arabic,  
  
    English,  
  
    Russian,  
    Other  
  
}
```

Represents the language of signal intelligence content.

Abstract Base Class: Report

Purpose

Models the shared concept of an intelligence report regardless of source type.

Responsibility

- Holds all common report data
- Enforces validation through properties
- Provides abstract contract for source-specific behavior
- Manages report lifecycle state

Fields (Private)

- private int _reportId
- private DateTime _timestamp
- private double _latitude
- private double _longitude
- private string _description
- private ReportStatus _status
- private Priority _priority
- private Classification _classification
- private int _reliabilityScore
- private string _rejectionReason

Properties (Public)

- int ReportId { get; } - Read-only, set in constructor
- DateTime Timestamp { get; set; } - Validated: not in future, not before 2020-01-01
- double Latitude { get; set; } - Validated: 29.5000–33.5000 - double
- double Longitude { get; set; } - Validated: 34.0000–36.0000 - string
- string Description { get; set; } - Validated: 10–500 characters - ReportStatus
- ReportStatus Status { get; set; } - Validated: must be valid enum value - Priority
- Priority Priority { get; set; }
- Classification Classification { get; set; }
- int ReliabilityScore { get; set; } - Validated: 1–10
- string RejectionReason { get; set; }

Constructor (Protected)

protected Report(int reportId, DateTime timestamp, double latitude,
double longitude, string description)

- Visibility: `protected` (only derived classes can call)
- Initializes all common fields
- Sets `Status = ReportStatus.New`
- Validates input through properties

Abstract Methods

public abstract string GetSourceType();

public abstract int CalculateReliabilityScore();

Virtual Methods

public virtual string GetSummary()

Returns a formatted summary of the report. Derived classes can override for source-specific formatting.

Override Methods

```
public override string ToString()
```

Returns a complete string representation of the report.

Relationships

- **Inherited by:** DroneReport, SoldierReport, RadarReport, SignalReport -

Used by: ReportPipeline, ReportRepository, RejectedReportRepository

Derived Class: DroneReport

Purpose

Represents an intelligence report from a drone source.

Responsibility

- Stores drone-specific data (Altitude, ImageQuality) -
- Validates drone-specific fields
- Calculates reliability based on drone criteria

Fields (Private)

- `private int _altitude`
- `private int _imageQuality`

Properties (Public)

- `int Altitude { get; set; }` - Validated: 100–10000 -
- `int ImageQuality { get; set; }` - Validated: 1–100

Constructor

```
public DroneReport(int reportId, DateTime timestamp, double  
  
    latitude, double longitude, string description,  
  
    int altitude, int imageQuality)
```

```
: base(reportId, timestamp, latitude, longitude, description)
```

- Calls base constructor with common fields
- Initializes drone-specific fields through properties

Override Methods

```
public override string GetSourceType() => "Drone";
```

```
public override int CalculateReliabilityScore()
```

Implements drone-specific reliability calculation:

- Base: 5
- ImageQuality >= 80: +3
- ImageQuality >= 50: +2
- Altitude 500–3000: +2
- Altitude > 7000: -2
- Result clamped to 1–10

Relationships

- **Inherits from:** Report
- **Validated by:** DroneValidator

Derived Class: SoldierReport

Purpose

Represents an intelligence report from a soldier in the field.

Responsibility

- Stores soldier-specific data (SoldierName, SoldierID, Unit, ConfidenceLevel) - Validates soldier-specific fields
- Calculates reliability based on soldier criteria

Fields (Private)

- `private string _soldierName`
- `private string _soldierID`
- `private string _unit`
- `private int _confidenceLevel`

Properties (Public)

- `string SoldierName { get; set; }` - Validated: 2–50 characters -
- `string SoldierID { get; set; }` - Validated: exactly 7 digits -
- `string Unit { get; set; }` - Validated: 2–50 characters - `int`
- `ConfidenceLevel { get; set; }` - Validated: 1–5

Constructor

```
public SoldierReport(int reportId, DateTime timestamp, double latitude,
                    double longitude, string description,

                    string soldierName, string soldierID, string unit,

                    int confidenceLevel)

    : base(reportId, timestamp, latitude, longitude, description)
```

Override Methods

```
public override string GetSourceType() => "Soldier";
```

```
public override int CalculateReliabilityScore()
```

Implements soldier-specific reliability calculation:

- Base: 4
- Add ConfidenceLevel value
- Description contains weapon/vehicle/movement/explosion: +1 -
- Result clamped to 1–10

Relationships

- **Inherits from:** Report
- **Validated by:** SoldierValidator

Derived Class: RadarReport

Purpose

Represents an intelligence report from a radar system.

Responsibility

- Stores radar-specific data (Speed, Direction, Distance) -
- Validates radar-specific fields
- Calculates reliability based on radar criteria

Fields (Private)

- `private int _speed`
- `private int _direction`
- `private int _distance`

Properties (Public)

- `int Speed { get; set; }` - Validated: 0–2000 - `int Direction { get; set; }` - Validated: 0–360 - `int Distance { get; set; }` - Validated: 100–100000

Constructor

```
public RadarReport(int reportId, DateTime timestamp, double
    latitude, double longitude, string description,
    int speed, int direction, int distance)
: base(reportId, timestamp, latitude, longitude, description)
```

Override Methods

```
public override string GetSourceType() => "Radar";

public override int CalculateReliabilityScore()
```

Implements radar-specific reliability calculation:

- Base: 6
- Distance 500–30000: +2
- Speed 10–900: +1
- Distance > 70000: -2
- Speed > 1500: -2
- Result clamped to 1–10

Relationships

- **Inherits from:** Report
- **Validated by:** RadarValidator

Derived Class: SignalReport

Purpose

Represents an intelligence report from a signal intelligence system.

Responsibility

- Stores signal-specific data (Frequency, Content, Language, SignalStrength) - Validates signal-specific fields
- Calculates reliability based on signal criteria

Fields (Private)

- `private double _frequency`
- `private string _content`

```
-private Language _language  
-private int _signalStrength
```

Properties (Public)

```
- double Frequency { get; set; } - Validated: 1.0–3000.0 -  
string Content { get; set; } - Validated: 5–1000 characters -  
Language Language { get; set; }  
- int SignalStrength { get; set; } - Validated: -120–0
```

Constructor

```
public SignalReport(int reportId, DateTime timestamp, double latitude,  
  
                    double longitude, string description,  
  
                    double frequency, string content, Language language,  
  
                    int signalStrength)  
  
: base(reportId, timestamp, latitude, longitude, description)
```

Override Methods

```
public override string GetSourceType() => "Signal";
```

```
public override int CalculateReliabilityScore()  
Implements signal-specific reliability calculation:
```

```
- Base: 5  
- SignalStrength >= -40: +3  
- SignalStrength >= -70: +2  
- Content contains attack/target/border/vehicle: +1 -  
SignalStrength < -100: -2  
- Result clamped to 1–10
```

Relationships

```
- Inherits from: Report  
- Validated by: SignalValidator
```

Interface: IValidator

Purpose

Defines the contract for report validation.

Methods

ValidationResult Validate(Report report);

Implementing Classes

- DroneValidator
- SoldierValidator
- RadarValidator
- SignalValidator

Class: ValidationResult

Purpose

Encapsulates the result of a validation operation.

Responsibility

- Indicates whether validation passed or failed -
- Stores the reason for failure if applicable

Fields (Private)

- `private bool _isValid`
- `private string _errorMessage`

Properties (Public)

- `bool IsValid { get; }`
- `string ErrorMessage { get; }`

Constructors

`public ValidationResult(bool isValid, string errorMessage)`

Static Factory Methods

`public static ValidationResult Success()`

`public static ValidationResult Failure(string errorMessage)`

Relationships

- **Returned by:** All IValidator implementations -
- Used by:** ReportPipeline

Class: DroneValidator

Purpose

Validates drone-specific report fields.

Responsibility

- Implements IValidator for DroneReport
- Checks all drone-specific validation rules

Methods (Public)

public ValidationResult Validate(Report report)

Validates:

- Report is DroneReport type
- Altitude: 100–10000
- ImageQuality: 1–100
- All common fields (inherited validation)

Relationships

- **Implements:** IValidator
- **Validates:** DroneReport

Class: SoldierValidator

Purpose

Validates soldier-specific report fields.

Responsibility

- Implements IValidator for SoldierReport -
- Checks all soldier-specific validation rules

Methods (Public)

public ValidationResult Validate(Report report)

Validates:

- Report is SoldierReport type
- SoldierName: 2–50 characters
- SoldierID: exactly 7 digits
- Unit: 2–50 characters
- ConfidenceLevel: 1–5
- All common fields

Relationships

- **Implements:** IValidator
- **Validates:** SoldierReport

Class: RadarValidator

Purpose

Validates radar-specific report fields.

Responsibility

- Implements IValidator for RadarReport -
- Checks all radar-specific validation rules

Methods (Public)

```
public ValidationResult Validate(Report report)
```

Validates:

- Report is RadarReport type
- Speed: 0–2000
- Direction: 0–360
- Distance: 100–100000
- All common fields

Relationships

- **Implements:** IValidator
- **Validates:** RadarReport

Class: SignalValidator

Purpose

Validates signal-specific report fields.

Responsibility

- Implements IValidator for SignalReport
- Checks all signal-specific validation rules

Methods (Public)

public ValidationResult Validate(Report report)

Validates:

- Report is SignalReport type
- Frequency: 1.0–3000.0
- Content: 5–1000 characters
- Language: valid enum value
- SignalStrength: -120–0
- All common fields

Relationships

- **Implements:** IValidator
- **Validates:** SignalReport

Class: ReliabilityCalculator

Purpose

Centralized location for reliability score calculation logic.

Responsibility

- Delegates to each report's CalculateReliabilityScore method -
- Ensures score is within valid range

Methods (Public)

public int Calculate(Report report)

Calls `report.CalculateReliabilityScore()` and ensures result is 1–10.

Relationships

- **Uses:** Report (polymorphically)
- **Used by:** ReportPipeline

Class: PriorityCalculator

Purpose

Calculates report priority based on business rules.

Responsibility

- Analyzes report content and source type
- Applies priority determination rules
- Returns appropriate Priority enum value

Methods (Public)

public Priority Calculate(Report report)

Implements priority rules:

- Critical: missile/explosion/attack/fire keywords, Radar Speed ≥ 800 , Signal with target AND attack
- High: weapon/suspicious/border keywords, Drone Altitude < 500 , Radar Speed ≥ 400 , Soldier ConfidenceLevel ≥ 4 with movement
- Medium: movement/vehicle/activity keywords, Radar Speed ≥ 120 , ReliabilityScore ≥ 7
- Low: default

Private Helper Methods

private bool ContainsKeyword(string text, params string[] keywords)

Case-insensitive keyword search.

Relationships

- **Uses:** Report (polymorphically), DroneReport, RadarReport, SoldierReport, SignalReport
- **Used by:** ReportPipeline

Class: ClassificationCalculator

Purpose

Calculates security classification based on business rules.

Responsibility

- Analyzes report priority and content
- Applies classification determination rules
- Returns appropriate Classification enum value

Methods (Public)

public Classification Calculate(Report report)

Implements classification rules:

- TopSecret: Priority=Critical OR Content contains target/attack/missile - Secret: Priority=High OR SourceType=Signal OR Description contains weapon/border - Restricted: Priority=Medium OR SourceType=Soldier
- Unclassified: default

Private Helper Methods

private bool ContainsKeyword(string text, params string[] keywords)

Relationships

- **Uses:** Report (polymorphically), SignalReport, SoldierReport
- **Used by:** ReportPipeline

Class: ReportRepository

Purpose

Stores and manages validated reports.

Responsibility

- Maintains collection of validated reports
- Provides search, filter, and retrieval operations -
- Manages report status updates

Fields (Private)

- `private List<Report> _reports`

Constructor

public ReportRepository()

Initializes empty report list.

Methods (Public)

public void Add(Report report)

public List<Report> GetAll()

```
public List<Report> GetByStatus(ReportStatus status)

public List<Report> GetByPriority(Priority priority)

public List<Report> Search(string keyword)

public Report GetById(int reportId)

public void UpdateStatus(int reportId, ReportStatus newStatus)

public int GetTotalCount()

public int GetCountByStatus(ReportStatus status)
```

Relationships

- **Stores:** Report (and all derived types)
- **Used by:** ReportPipeline, Program

Class: RejectedReportRepository

Purpose

Stores and manages rejected reports separately.

Responsibility

- Maintains collection of rejected reports
- Provides access to rejection reasons
- Enables analysis of validation failures

Fields (Private)

```
- private List<Report> _rejectedReports
```

Constructor

```
public RejectedReportRepository()
```

Methods (Public)

```
public void Add(Report report)
```

```
public List<Report> GetAll()
```

```
public int GetTotalCount()
```

```
public List<Report> GetByReason(string reasonKeyword)
```

Relationships

- **Stores:** Report (rejected instances)
- **Used by:** ReportPipeline, Program

Class: ReportPipeline

Purpose

Orchestrates the complete report processing workflow.

Responsibility

- Receives incoming reports
- Executes validation
- Calculates reliability, priority, classification
- Routes to appropriate repository
- Manages report lifecycle

Fields (Private)

```
- private ReportRepository _validatedReports -  
private RejectedReportRepository _rejectedReports -  
private int _nextReportId
```

Constructor

```
public ReportPipeline()
```

Initializes repositories and ID counter.

Methods (Public)

```
public void ProcessReport(Report report)
```

```
public ReportRepository GetValidatedReports()
```

```
public RejectedReportRepository GetRejectedReports()
```

```
public void DisplayStatistics()
```

Private Methods

```
private IValidator GetValidator(Report report)
```

```
private void ValidateReport(Report report)
```

```
private void CalculateMetrics(Report report)
```

```
private void StoreReport(Report report)
```

Processing Flow

1. Receive Report (Status = New)
2. Set Status = Validating
3. Get appropriate validator
4. Execute validation
5. If validation fails:
 - Set Status = Rejected
 - Set RejectionReason
 - Store in RejectedReportRepository
 - END
6. If validation succeeds:
 - Set Status = Validated
 - Calculate ReliabilityScore
 - Calculate Priority
 - Calculate Classification
 - Store in ReportRepository

Relationships

- **Uses:** Report, IValidator, DroneValidator, SoldierValidator, RadarValidator, SignalValidator, ReliabilityCalculator, PriorityCalculator, ClassificationCalculator, ReportRepository, RejectedReportRepository
- **Used by:** Program

Class: Program

Purpose

Application entry point and user interface.

Responsibility

- Creates sample reports
- Demonstrates pipeline processing
- Displays results and statistics

Methods (Public/Static)

```
public static void Main(string[] args)
```

Private Helper Methods

```
private static void DisplayReport(Report report)
```

```
private static void DisplayValidatedReports(ReportRepository repository) private
```

```
static void DisplayRejectedReports(RejectedReportRepository repository)
```

Relationships

- **Uses:** All report types, ReportPipeline, ReportRepository, RejectedReportRepository

Object Communication Flow

Program

↓

Creates Report instances (DroneReport, SoldierReport, etc.)

↓

Passes to → ReportPipeline.ProcessReport()

↓

Status = New

↓

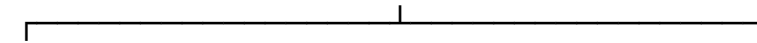
Status = Validating

↓

GetValidator() → Returns appropriate IValidator



Validator.Validate() → Returns ValidationResult



IsValid = false IsValid = true



Status = Rejected Status = Validated



Set RejectionReason ReliabilityCalculator.Calculate() ↓ ↓

Store in PriorityCalculator.Calculate()

RejectedReportRepository ↓

ClassificationCalculator.Calculate()



Store in ReportRepository



Retrieves results



Displays to console

Relationships Summary

Inheritance Hierarchy

Report (abstract)

└─ DroneReport

└─ SoldierReport

└─ RadarReport

└─ SignalReport

Interface Implementation

IValidator (interface)

└─ DroneValidator

└─ SoldierValidator

└─ RadarValidator

└─ SignalValidator

Composition

- ReportPipeline **contains** ReportRepository
- ReportPipeline **contains** RejectedReportRepository -
ReportRepository **contains** List
- RejectedReportRepository **contains** List

Dependencies

- ReportPipeline **depends on** all validators
- ReportPipeline **depends on** all calculators
- Validators **depend on** specific report types
- Calculators **depend on** Report (polymorphically) - Program
depends on ReportPipeline and all report types